

CYBER SECURITY

PROJECT 2

Basic Encryption & Decryption Using a Caesar-Style Shift Cipher

Submitted By : Maryam Malick

Internship : Cyber Security

Technology : Python

Environment : VS Code

Provider : DecodeLabs

Batch : 2026

1. Executive Summary

This report covers Project 2 of the DecodeLabs Cybersecurity Internship: Basic Encryption & Decryption. The task was to build a small Python program that can take a message typed by a user, scramble (encrypt) it so it cannot be read directly, and then reverse the process (decrypt) to bring back the original message.

The program was written and tested in Visual Studio Code. It uses a fixed shift value of 3, in the spirit of the classic Caesar Cipher, and shifts every character of the message using Python's built-in `ord()` and `chr()` functions. Testing showed that the decrypted output always matched the original message exactly, confirming that the encryption and decryption logic works correctly.

2. Project Objective

The objective is to implement a simple, working encryption and decryption technique using Python, and to display the original message, its encrypted form, and the decrypted form so the process can be clearly verified.

3. Requirements Covered

- Encrypt the user's text using a basic logic (Caesar Cipher or similar).
- Decrypt the encrypted text back to its original form.
- Display both the encrypted and the decrypted output.
- Demonstrate the working program through practical testing.

4. Tools and Environment

Item	Details
Programming Language	Python 3
Code Editor	Visual Studio Code
Core Concept	Caesar-style Cipher (fixed shift of 3)
Operating System	Windows

5. Concept Used

A Caesar Cipher normally works by moving each letter of a message a fixed number of steps along the alphabet. In this implementation, the same idea is applied at the character-code level instead of the alphabet level: every character in the message is converted to its ASCII number using `ord()`, that number is increased by a fixed shift of 3 using `chr()` to turn it back into a character. Because the shift is applied to the numeric code of every character (not only letters), spaces, digits, and punctuation are also transformed as part of the process, and are correctly restored during decryption since the same shift is simply subtracted.

6. Program Design

The program is intentionally kept short and beginner-friendly. It uses a fixed key (shift = 3) rather than asking the user to choose one, which keeps the logic simple and easy to follow while still demonstrating the core idea of a shift cipher: the same key that scrambles the message is also used to unscramble it.

The program runs in two clear stages inside the same script: first it builds the encrypted text character by character, then it runs the reverse loop on the encrypted text to rebuild the decrypted text.

7. Algorithm

7.1 Encryption Steps

1. Ask the user to enter a message.
2. Set the shift value to 3.
3. Create an empty string to hold the encrypted result.
4. Go through the message one character at a time.
5. Convert the character to its ASCII code using `ord()`, add the shift, and convert it back to a character using `chr()`.
6. Add this new character to the encrypted result.
7. Print the final encrypted text.

7.2 Decryption Steps

1. Create an empty string to hold the decrypted result.
2. Go through the encrypted text one character at a time.
3. Convert the character to its ASCII code using `ord()`, subtract the same shift, and convert it back to a character using `chr()`.
4. Add this new character to the decrypted result.
5. Print the final decrypted text and confirm it matches the original message.

8. Python Program

The complete program used for this project is shown below (the full listing also appears in the Appendix):

```
message = input("Enter your message: ")
shift = 3
encrypted = ""

for letter in message:
    new_letter = ord(letter) + shift
    encrypted += chr(new_letter)

print("Encrypted Text:", encrypted)

decrypted = ""

for letter in encrypted:
    new_letter = ord(letter) - shift
    decrypted += chr(new_letter)

print("Decrypted Text:", decrypted)
```

9. Testing and Results

The program logic was tested with several different messages, including plain words, mixed-case sentences, numbers, and symbols, to confirm that decryption always restores the original message.

Message	Encrypted Output	Decrypted Output
hello	khoor	hello
DecodeLabs Internship	Ghfrgh0dev#Lqwhuqvkl\$	DecodeLabs Internship
Cyber Security	F ehu#Vhfxulw	Cyber Security
Maryam 2026!	Pdu dp#5359\$	Maryam 2026!

In every test case, the decrypted output matched the original message exactly, confirming that the encryption and decryption logic is correct and fully reversible.

10. Evidence and Screenshot

The screenshot below shows the program running successfully in Visual Studio Code, encrypting the message "hello" into "khoor" and then decrypting it back to "hello":

```

1 message = input("Enter your message: ")
2 shift = 3
3 encrypted = ""
4
5 for letter in message:
6     new_letter = ord(letter) + shift
7     encrypted += chr(new_letter)
8
9 print("Encrypted Text:", encrypted)
10
11 decrypted = ""
12
13 for letter in encrypted:
14     new_letter = ord(letter) - shift
15     decrypted += chr(new_letter)
16
17 print("Decrypted Text:", decrypted)
18

```

```

PS C:\Users\PMLS\Desktop> cd DecodeLabs-Internship
PS C:\Users\PMLS\Desktop\DecodeLabs-Internship> cd Project-2-Caesar-Cipher
PS C:\Users\PMLS\Desktop\DecodeLabs-Internship\Project-2-Caesar-Cipher> python main.py
Enter your message: hello
Encrypted Text: koor
Decrypted Text: hello
PS C:\Users\PMLS\Desktop\DecodeLabs-Internship\Project-2-Caesar-Cipher>

```

Figure 1: Program output in VS Code for the message "hello"

11. Observations

- Every character typed by the user, including letters, was shifted correctly by the ASCII-based method.
- Because the shift is applied to the raw character code rather than only to letters, spaces, digits, and punctuation are also transformed in the encrypted text (for example, the space in "DecodeLabs Internship" became the # symbol). This does not cause any problem, since the same shift used to encrypt is simply reversed to decrypt, so the original message is always fully restored.
- The decrypted text matched the original message in every test performed.
- Using a fixed shift of 3 keeps the code short and easy to understand, though it means the same key is used every time the program runs.

12. Limitations and Future Enhancements

12.1 Limitations

Because the shift is fixed at 3 and is not based on the standard 26-letter wraparound used in a classic Caesar Cipher, encrypting a letter near the end of the alphabet (or certain punctuation marks) can produce an unusual or non-printable character in the encrypted text. This does not break the program, since decryption reverses it correctly, but it means the encrypted output is not restricted to only letters, unlike a traditional Caesar Cipher. In addition, since the key is fixed in the code, anyone who reads the program can immediately work out how to decrypt any message, which makes this approach unsuitable for protecting real, sensitive information.

12.2 Future Enhancements

- Allow the user to enter their own shift key instead of using a fixed value of 3.
- Restrict the shifting logic to only affect letters (A–Z, a–z) using `isalpha()`, while leaving spaces, numbers, and symbols unchanged, matching a traditional Caesar Cipher more closely.
- Upgrade the technique to a Vigenère Cipher, which uses a repeating keyword instead of a single fixed number for stronger protection.
- Add input validation and error handling for unexpected input.

13. Conclusion

This project successfully implemented a basic encryption and decryption program in Python using an ASCII shift technique inspired by the Caesar Cipher. The program correctly encrypted user-provided messages and, using the same fixed key, decrypted them back to their original form. Testing with different words, sentences, numbers, and symbols confirmed that the program worked reliably every time. Overall, this project provided practical, hands-on experience with basic cryptographic logic, string handling, and the general idea of how a shared key can be used to both lock and unlock data.

14. References

- [1] Python Software Foundation. Python 3 Documentation — `ord()` and `chr()` functions.
- [2] NIST Computer Security Resource Center. Glossary — Encryption.
- [3] NIST Computer Security Resource Center. Glossary — Cryptographic Algorithm.
- [4] DecodeLabs. Industrial Training Kit — Cyber Security Project 2, Batch 2026.

Appendix A — Full Python Source Code

```
message = input("Enter your message: ")
shift = 3
encrypted = ""

for letter in message:
    new_letter = ord(letter) + shift
    encrypted += chr(new_letter)

print("Encrypted Text:", encrypted)

decrypted = ""

for letter in encrypted:
    new_letter = ord(letter) - shift
    decrypted += chr(new_letter)

print("Decrypted Text:", decrypted)
```